

Spring Boot

Java Backend Interview — Short Notes

1. Spring Boot Overview

- Opinionated, convention-over-configuration framework built on Spring. Removes boilerplate setup
- **Key goals:** standalone JAR, embedded server, auto-configuration, production-ready (Actuator), no XML
- **@SpringBootApplication** = @Configuration + @ComponentScan + @EnableAutoConfiguration

```
@SpringBootApplication
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

- **SpringApplication.run()** steps: detect app type → load context initializers → create ApplicationContext → refresh (scan, auto-config, init beans) → call ApplicationRunners/CommandLineRunners

Feature	Description	Config
Embedded Server	Tomcat (default), Jetty, Undertow. No WAR deployment needed	spring-boot-starter-web (Tomcat) or starter-jetty
Auto-Configuration	Detect classpath, configure beans automatically	@EnableAutoConfiguration, spring.autoconfigure.exclude
Starter POMs	Curated dependency sets — add one starter, get compatible deps	spring-boot-starter-*, spring-boot-starter-test
Actuator	Production monitoring — health, metrics, info, env, beans	management.endpoints.web.exposure.include=*
Externalized Config	application.yml/properties, env vars, command-line args	spring.config.location, SPRING_APPLICATION_JSON
DevTools	Hot reload, LiveReload, faster restarts in development	spring-boot-devtools dependency (scope: runtime)

2. Auto-Configuration Deep Dive

- Classes registered in META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports
- Each class uses @Conditional annotations — activates only when conditions met
- Debug: run with --debug flag or set logging.level.org.springframework.boot.autoconfigure=DEBUG

Conditional	Activates When	Example
@ConditionalOnClass	Class exists on classpath	@ConditionalOnClass(DataSource.class)
@ConditionalOnMissingClass	Class NOT on classpath	If Tomcat not present, use Undertow
@ConditionalOnBean	Bean of type exists in context	@ConditionalOnBean(AuthenticationManager.class)

@ConditionalOnMissingBean	No bean of type in context — user hasn't defined one	Auto-config DataSource only if user hasn't defined one
@ConditionalOnProperty	Property has specified value	@ConditionalOnProperty(name="cache.type",havingValue="redis")
@ConditionalOnWebApplication	Is a web application context	Enable MVC auto-config only in web context
@ConditionalOnExpression	SpEL expression is true	@ConditionalOnExpression("\${feature.enabled:false}")
@ConditionalOnResource	Resource exists on classpath	@ConditionalOnResource(resources="classpath:schema.sql")

```
// Custom auto-configuration
@Configuration
@ConditionalOnClass(MyLibrary.class)
@ConditionalOnMissingBean(MyService.class)
public class MyAutoConfiguration {
    @Bean public MyService myService() { return new MyService(); }
}

// Register in: META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports
```

3. Spring Boot Starters

Starter	Includes	Key Beans
spring-boot-starter-web	Spring MVC, Tomcat, Jackson	DispatcherServlet, ObjectMapper, Tomcat
spring-boot-starter-webflux	Spring WebFlux, Reactor, Netty	RouterFunction, WebClient, Flux, Mono
spring-boot-starter-data-jpa	Spring Data JPA, Hibernate, HikariCP	JpaRepository, EntityManager, DataSource
spring-boot-starter-data-mongodb	Spring Data MongoDB, MongoDB driver	MongoRepository, MongoTemplate
spring-boot-starter-data-redis	Lettuce/Jedis, Spring Data Redis	RedisTemplate, StringRedisTemplate
spring-boot-starter-security	Spring Security	SecurityFilterChain, AuthenticationManager
spring-boot-starter-test	JUnit 5, Mockito, AssertJ, MockMvc	@SpringBootTest, @MockBean, @WebMvcTest
spring-boot-starter-actuator	Micrometer, health indicators	/actuator/health, /actuator/metrics
spring-boot-starter-validation	Hibernate Validator, Jakarta Validation	@Valid, @NotNull, @Size, ConstraintViolation
spring-boot-starter-cache	Spring Cache abstraction	@Cacheable, @CacheEvict, CacheManager
spring-boot-starter-aop	Spring AOP, AspectJ	@Aspect, @Before, @Around
spring-boot-starter-kafka	Spring Kafka, Kafka client	KafkaTemplate, @KafkaListener

4. Configuration & Properties

4.1 Property Sources (Priority — highest to lowest)

Priority	Source	Example
1 (Highest)	Command-line args	--server.port=9090
2	SPRING_APPLICATION_JSON env var / system property	SPRING_APPLICATION_JSON='{ "server.port": 9090 }'
3	OS environment variables	SERVER_PORT=9090 (relaxed binding)
4	application-{profile}.properties outside jar	application-prod.properties
5	application.properties outside jar	./config/application.properties
6	application-{profile}.properties inside jar	src/main/resources/application-prod.yml
7 (Lowest)	application.properties inside jar	src/main/resources/application.yml

4.2 @ConfigurationProperties

```
@ConfigurationProperties(prefix = "app.mail")
@Validated // enable bean validation
public class MailProperties {
    @NotBlank private String host;
    @Min(1) @Max(65535) private int port = 587;
    private boolean tls = true;
    // getters/setters
}

# application.yml
app.mail.host: smtp.gmail.com
app.mail.port: 587

// Enable: @EnableConfigurationProperties(MailProperties.class)
// Or: @ConfigurationPropertiesScan on main class
```

4.3 Profile-specific Config

```
# application.yml – active for all profiles
spring.application.name: order-service
server.port: 8080
---
spring.config.activate.on-profile: prod
spring.datasource.url: jdbc:postgresql://prod-db/orders
logging.level.root: WARN
---
spring.config.activate.on-profile: dev
spring.datasource.url: jdbc:h2:mem:testdb
logging.level.com.example: DEBUG
```

5. Spring MVC & REST

5.1 Key MVC Annotations

Annotation	Purpose	Example
@RestController	@Controller + @ResponseBody — return JSON/XML	@RestController public class OrderController
@RequestMapping	Map URL patterns to class or method	@RequestMapping("/api/orders")

@GetMapping/@PostMapping	Shorthand for @RequestMapping(method=GET/POST)	@GetMapping("/{id}")
@PathVariable	Extract value from URI path	@PathVariable Long id
@RequestParam	Extract query parameter	@RequestParam(defaultValue="0") int page
@RequestBody	Deserialize JSON request body to object	@RequestBody @Valid OrderRequest req
@ResponseBody	Serialize return value to response body	Included in @RestController
@ResponseStatus	Set HTTP status code on method or class	@ResponseStatus(HttpStatus.CREATED)
@Valid / @Validated	Trigger bean validation on request body/params	@RequestBody @Valid CreateUserRequest req
@ExceptionHandler	Handle specific exception in controller	@ExceptionHandler(UserNotFoundException.class)
@ModelAttribute	Bind request params to model object (MVC)	@ModelAttribute UserForm form
@CrossOrigin	Enable CORS for controller or method	@CrossOrigin(origins="https://frontend.com")

5.2 Request Lifecycle in Spring MVC

- **DispatcherServlet** → HandlerMapping (find controller) → HandlerAdapter (invoke method) → MessageConverter (serialize) → Response
- Filters run BEFORE DispatcherServlet (security, logging, CORS). Interceptors run inside DispatcherServlet (preHandle/postHandle/afterCompletion)

5.3 ResponseEntity

```

@PostMapping("/orders")
public ResponseEntity createOrder(@RequestBody @Valid OrderRequest req) {
    Order order = orderService.create(req);
    URI location = URI.create("/orders/" + order.getId());
    return ResponseEntity
        .created(location) // 201 Created + Location header
        .body(new OrderResponse(order));
}

```

5.4 Bean Validation

Annotation	Description	Example
@NotNull	Field must not be null	@NotNull String name
@NotBlank	String not null AND not empty (trimmed)	@NotBlank String email
@NotEmpty	Collection/String not null AND not empty	@NotEmpty List tags
@Size(min,max)	String/Collection length in range	@Size(min=2,max=50) String username
@Min/@Max	Numeric value range	@Min(1) @Max(100) int quantity
@Email	Valid email format	@Email String email
@Pattern	Matches regex	@Pattern(regexp="[A-Z]{2}\\d{4}") String code

@Positive/@Negative	Number > 0 or < 0	@Positive BigDecimal amount
@Past/@Future	Date in past or future	@Past LocalDate birthDate
@Valid	Cascade validation to nested objects	@Valid Address address

6. Spring Boot Actuator

- Production-ready features — health checks, metrics, monitoring, env inspection
- Expose endpoints via HTTP (/actuator/*) or JMX. Secure non-health endpoints in production

Endpoint	Description	Key Data
/actuator/health	Service health + dependency health (DB, Redis, Kafka)	status: UP/DOWN, components map. Liveness & readiness probes
/actuator/metrics	Micrometer metrics — JVM, HTTP, custom	jvm.memory.used, http.server.requests, hikaricp.connections
/actuator/info	App info — version, git commit, build	Populate via info.* properties or build-info plugin
/actuator/env	All environment properties and their sources	Shows active profiles, property sources, values
/actuator/beans	All Spring beans in context	Bean name, type, scope, dependencies
/actuator/mappings	All @RequestMapping endpoints	URL patterns, methods, handler classes
/actuator/loggers	View and change log levels at runtime	POST to change: {"configuredLevel": "DEBUG"}
/actuator/threaddump	Current thread states	Deadlock detection, thread analysis
/actuator/heapdump	Download heap dump — memory analysis	Open with Eclipse MAT, VisualVM
/actuator/prometheus	Prometheus-format metrics scrape endpoint	Add micrometer-registry-prometheus
/actuator/refresh	Refresh @RefreshScope beans (Spring Cloud)	POST to reload config without restart
/actuator/shutdown	Gracefully shut down application	Disabled by default. Enable explicitly

```
# application.yml – expose all endpoints (secure in prod!)
management:
  endpoints.web.exposure.include: "*"
  endpoint.health.show-details: always
  endpoint.health.probes.enabled: true # K8s liveness/readiness

# Custom health indicator
@Component
public class ExternalApiHealthIndicator implements HealthIndicator {
  @Override public Health health() {
    try { externalApi.ping(); return Health.up().build(); }
    catch(Exception e) { return Health.down().withException(e).build(); }
```

```

}
}

```

7. Testing in Spring Boot

7.1 Testing Annotations

Annotation	What it Loads	Use For
@SpringBootTest	Full application context. Embedded server optional	Integration tests — test full stack, real beans
@WebMvcTest(Ctrl)	Only MVC layer — controller, filters, @ControllerAdvice	Unit test controller — mock service layer with @MockBean
@DataJpaTest	JPA layer — repositories, EntityManager, in-memory H2	Repository tests without full context
@DataMongoTest	MongoDB repositories — embedded MongoDB	MongoDB repository tests
@JsonTest	Jackson JSON serialization/deserialization	Test @JsonSerialize/@JsonDeserialize
@RestClientTest	Rest client (RestTemplate/WebClient) + MockServer	Test service that calls external REST API
@MockBean	Add Mockito mock into Spring context	Mock service in @WebMvcTest. Replaces real bean
@SpyBean	Spy (real bean + mock methods) in Spring context	Verify calls on real bean
@TestConfiguration	Additional @Bean defs for test only	Provide test doubles without @MockBean
@ActiveProfiles	Activate profiles for test	@ActiveProfiles("test") — loads test config
@Sql / @SqlGroup	Execute SQL scripts before/after test	Load test data, clean up after

7.2 MockMvc — Controller Testing

```

@WebMvcTest(OrderController.class)
class OrderControllerTest {
    @Autowired MockMvc mockMvc;
    @MockBean OrderService orderService; // mock the service

    @Test void createOrder_returns201() throws Exception {
        given(orderService.create(any())).willReturn(new Order(1L, "PENDING"));
        mockMvc.perform(post("/api/orders")
            .contentType(APPLICATION_JSON)
            .content("{\"item\":\"book\",\"qty\":1}"))
            .andExpect(status().isCreated())
            .andExpect(jsonPath("$.id").value(1))
            .andExpect(jsonPath("$.status").value("PENDING"));
    }
}

```

7.3 Integration Test with @SpringBootTest

```

@SpringBootTest(webEnvironment = WebEnvironment.RANDOM_PORT)
@ActiveProfiles("test")
class OrderIntegrationTest {

```

```

@Autowired TestRestTemplate restTemplate;
@Autowired OrderRepository orderRepository;

@Test void placeAndRetrieveOrder() {
    ResponseEntity resp = restTemplate
        .postForEntity("/api/orders", new OrderRequest("book",1), Order.class);
    assertThat(resp.getStatusCode()).isEqualTo(CREATED);
    assertThat(orderRepository.findById(resp.getBody().getId()).isPresent());
}
}

```

8. Caching

- Spring Cache abstraction — @EnableCaching + cache annotations. Backend: Caffeine, Redis, EhCache, Hazelcast

Annotation	Description	Example
@Cacheable	Cache result. On subsequent calls with same key, return cached value	@Cacheable(value="users", key="#id")
@CachePut	Always execute method AND update cache. For write-through	@CachePut(value="users", key="#user.id")
@CacheEvict	Remove entry or entire cache. allEntries=true clears all	@CacheEvict(value="users", key="#id")
@Caching	Combine multiple cache annotations on one method	@Caching(evict={@CacheEvict("users"), @CacheEvict("roles")})
@EnableCaching	Enable Spring cache on @Configuration class	@EnableCaching on @SpringBootApplication class

```

# application.yml – Redis cache
spring.cache.type: redis
spring.cache.redis.time-to-live: 600000 # 10 minutes

@Service
public class UserService {
    @Cacheable(value="users", key="#id", unless="#result == null")
    public User findById(Long id) { return userRepo.findById(id).orElse(null); }

    @CacheEvict(value="users", key="#user.id")
    public User update(User user) { return userRepo.save(user); }
}

```

9. Spring Security Fundamentals

- Filter-chain based security. SecurityFilterChain processes every request before reaching controller
- Key filters: UsernamePasswordAuthenticationFilter, JwtAuthenticationFilter (custom), ExceptionTranslationFilter

```

@Configuration @EnableWebSecurity @EnableMethodSecurity
public class SecurityConfig {
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http
            .csrf(csrf -> csrf.disable()) // disable for REST APIs
            .sessionManagement(sm -> sm.sessionCreationPolicy(STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers("/actuator/health").permitAll()
                .requestMatchers(HttpMethod.GET, "/api/products/**").hasRole("USER")
            )
        ;
    }
}

```

```

.anyRequest().authenticated()
.addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
.build();
}
}

// Method-level security
@PreAuthorize("hasRole('ADMIN')")
public void deleteUser(Long id) { ... }
@PreAuthorize("#userId == authentication.principal.id or hasRole('ADMIN')")
public User getUser(Long userId) { ... }

```

10. Async Processing

- **@EnableAsync** on config class. **@Async** on method — runs in separate thread pool
- Returns void (fire-and-forget) or Future/CompletableFuture (for results)
- Default executor: SimpleAsyncTaskExecutor (no pooling!). Configure ThreadPoolTaskExecutor
- **@Async has same self-invocation problem as AOP — must call from different bean**

```

@Configuration @EnableAsync
public class AsyncConfig {
    @Bean("taskExecutor")
    public Executor taskExecutor() {
        ThreadPoolTaskExecutor exec = new ThreadPoolTaskExecutor();
        exec.setCorePoolSize(4);
        exec.setMaxPoolSize(8);
        exec.setQueueCapacity(100);
        exec.setThreadNamePrefix("async-");
        return exec;
    }
}

@Service
public class EmailService {
    @Async("taskExecutor")
    public CompletableFuture sendEmail(String to) {
        // runs in taskExecutor thread pool
        return CompletableFuture.completedFuture("sent");
    }
}

```

11. Scheduling

- **@EnableScheduling** on config. **@Scheduled** on method
- Runs in single-threaded executor by default — configure TaskScheduler for parallel

Attribute	Description	Example
fixedRate	Run every N ms — from start of last execution	@Scheduled(fixedRate=5000) — every 5s
fixedDelay	Wait N ms AFTER last execution completes	@Scheduled(fixedDelay=5000)
initialDelay	Wait N ms before first execution	@Scheduled(initialDelay=10000, fixedRate=5000)
cron	Cron expression: sec min hour day month weekday	@Scheduled(cron="0 0 9 * * MON-FRI")

zone	Timezone for cron expressions	@Scheduled(cron="0 0 9 * * *", zone="Asia/Kolkata")
-------------	-------------------------------	---

```
@Scheduled(cron = "0 0 0 * * *") // midnight every day
public void cleanupExpiredSessions() { sessionRepo.deleteExpired(); }

@Scheduled(fixedRateString = "${app.cleanup.interval:60000}") // from property
public void pollQueue() { processNextBatch(); }
```

12. Filters, Interceptors & CORS

12.1 Filters vs Interceptors

	Filter (javax.servlet)	HandlerInterceptor (Spring MVC)
Level	Servlet container — before DispatcherServlet	Spring MVC — inside DispatcherServlet
Access to	Raw HttpServletRequest/Response	Handler (controller), ModelAndView
Spring beans	Limited — DelegatingFilterProxy to bridge	Full — part of Spring context
Methods	doFilter(request, response, chain)	preHandle, postHandle, afterCompletion
Use for	Cross-cutting: auth, logging, encoding, CORS	Business: auth check, logging, tracing headers

```
// Custom filter – add correlation ID to all requests
@Component
@Order(1) // filter order
public class CorrelationIdFilter extends OncePerRequestFilter {
    @Override protected void doFilterInternal(HttpServletRequest req,
        HttpServletResponse res, FilterChain chain) throws IOException, ServletException {
        String correlationId = Optional.ofNullable(req.getHeader("X-Correlation-Id"))
            .orElse(UUID.randomUUID().toString());
        MDC.put("correlationId", correlationId);
        res.setHeader("X-Correlation-Id", correlationId);
        try { chain.doFilter(req, res); } finally { MDC.clear(); }
    }
}
```

12.2 CORS Configuration

```
@Configuration
public class CorsConfig implements WebMvcConfigurer {
    @Override public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
            .allowedOrigins("https://app.example.com")
            .allowedMethods("GET", "POST", "PUT", "DELETE")
            .allowedHeaders("*")
            .allowCredentials(true)
            .maxAge(3600);
    }
}
```

13. Frequently Asked Interview Questions

Core & Auto-Config

Q1. What does `@SpringBootApplication` contain?

→ `@Configuration` + `@ComponentScan` (base package) + `@EnableAutoConfiguration`. Equivalent to writing all three. Can customize: exclude auto-configs, change scanBasePackages

Q2. How does auto-configuration work?

→ Auto-config classes listed in `AutoConfiguration.imports`. Spring loads them and evaluates `@Conditional` annotations. Only activates when conditions met (class on classpath, no user bean defined, property set). `--debug` shows condition evaluation report

Q3. How do you disable a specific auto-configuration?

→ `@SpringBootApplication(exclude={DataSourceAutoConfiguration.class})` or `spring.autoconfigure.exclude=...` in properties. Use when auto-config conflicts with your custom setup

Q4. What is the difference between `@SpringBootTest` and `@WebMvcTest`?

→ `@SpringBootTest`: loads full application context — integration tests, all beans, real DB. `@WebMvcTest`: loads only MVC layer (controller, filters) — unit test controllers with mocked services (`@MockBean`). `@WebMvcTest` is much faster

Q5. How do you create a custom Spring Boot Starter?

→ 1) Auto-config class with `@ConditionalOnXxx`. 2) Register in `AutoConfiguration.imports`. 3) Create starter pom with `spring-boot-autoconfigure`. 4) Optionally add `spring-boot-configuration-processor` for IDE metadata. Consumer adds your starter jar — auto-config activates automatically

Q6. What is `spring.main.lazy-initialization`?

→ `true` = all beans created on first use (not at startup). Faster startup, smaller memory footprint initially. Downside: errors discovered at first request not startup. Bad for production — use selectively with `@Lazy` instead

Q7. How do you read properties in Spring Boot?

→ `@Value("${key:default}")` for single values. `@ConfigurationProperties(prefix="app")` for groups of related properties — type-safe, validated. `Environment.getProperty()` programmatically. Prefer `@ConfigurationProperties` for related config

Q8. What is property relaxed binding?

→ Spring Boot maps kebab-case (`server-port`), `UPPER_SNAKE_CASE` (`SERVER_PORT`), camelCase (`serverPort`) all to same property. OS env vars in `UPPER_SNAKE` = standard in K8s/Docker config

MVC & REST

Q9. What is `DispatcherServlet`?

→ Front controller — single HTTP entry point. Routes requests to appropriate controllers via `HandlerMapping`. Invokes handler via `HandlerAdapter`. Uses `MessageConverter` to serialize/deserialize. Returns response

Q10. Difference between `@Controller` and `@RestController`?

→ `@RestController` = `@Controller` + `@ResponseBody`. All methods automatically serialize return value to JSON/XML. `@Controller`: returns view names (template rendering). Use `@RestController` for REST APIs

Q11. What does `@Valid` do? Where to place it?

→ Triggers JSR-380 bean validation on the annotated object. Place on `@RequestBody` parameter to validate incoming JSON. Place on `@RequestParam`, `@PathVariable` too. `MethodArgumentNotValidException` thrown if validation fails — handle in `@ControllerAdvice`

Q12. How do you handle exceptions globally?

→ `@RestControllerAdvice` with `@ExceptionHandler` methods. Applied to all controllers. Return `ResponseEntity` with appropriate status and `ErrorResponse` body. Spring Boot also provides `BasicErrorController` for /error fallback

Q13. What is content negotiation?

→ Spring MVC selects response format based on Accept header (`application/json`, `application/xml`) or URL extension (`.json`, `.xml`). Configured via `ContentNegotiationConfigurer`. Add `jackson-dataformat-xml` for XML support

Q14. How do you implement pagination?

→ Use `Pageable` as controller parameter + `Page` return. Spring Data repositories accept `Pageable` directly. Request: `GET /users?page=0&size=20&sort=name,asc`. Response includes content, `totalElements`, `totalPages`

Testing & Actuator

Q15. When to use @MockBean vs @Mock?

→ @MockBean: Spring context mock — replaces real bean in Spring context. Needed in @SpringBootTest and @WebMvcTest where Spring injects beans. @Mock (Mockito): pure unit test outside Spring context — faster, no context load

Q16. What does webEnvironment=RANDOM_PORT do?

→ Starts embedded server on random port — avoids port conflicts in CI. Use TestRestTemplate or WebClient to make real HTTP calls. Inject port with @LocalServerPort

Q17. How do you secure Actuator endpoints?

→ Expose only /actuator/health and /actuator/info publicly. Rest require authentication. In SecurityConfig: .requestMatchers("/actuator/health").permitAll() and .requestMatchers("/actuator/**").hasRole("ACTUATOR")

Q18. What is @Sql in tests?

→ Execute SQL scripts before/after test method or class. @Sql("classpath:test-data.sql") before. @Sql(executionPhase=AFTER_TEST_METHOD, scripts="cleanup.sql") after. Replaces @Before/@After with SQL setup/teardown

Advanced

Q19. Difference between @Async and @Scheduled?

→ @Async: marks method to run asynchronously in thread pool when called. @Scheduled: marks method to run automatically on timer/cron — not triggered by calling code. Both need @EnableAsync/@EnableScheduling

Q20. What is @RefreshScope?

→ Bean recreated when /actuator/refresh POST called. Used with Spring Cloud Config — live config updates. The bean proxy delegates to newly created instance. @ConfigurationProperties beans auto-refresh without @RefreshScope

Q21. How does Spring Boot handle graceful shutdown?

→ spring.lifecycle.timeout-per-shutdown-phase=30s. On SIGTERM: stop accepting new requests, wait for in-flight requests to complete (up to timeout), then shutdown. Embedded Tomcat/Netty support graceful shutdown. Enable: server.shutdown=graceful

Q22. What is CommandLineRunner and ApplicationRunner?

→ Interfaces with run() method called after context starts — for initialization logic. CommandLineRunner: receives String[] args. ApplicationRunner: receives ApplicationArguments (parsed). Both run before app is fully ready

Q23. What are Spring Boot Actuator's Micrometer metrics?

→ Micrometer is metrics facade — like SLF4J for metrics. Supports Prometheus, Datadog, CloudWatch. Auto-measures: JVM (heap, GC, threads), HTTP requests (rate, latency, status), HikariCP pool, Spring Cache, Kafka consumer lag

Q24. How do you configure multiple datasources?

→ Define two DataSource @Beans. Mark one @Primary. Use @Qualifier to inject non-primary. Separate JpaTransactionManager per datasource. Or use AbstractRoutingDataSource for routing. Disable DataSourceAutoConfiguration — configure manually

Q25. What is WebClient vs RestTemplate?

→ RestTemplate: synchronous, blocking. Legacy — still works but no active development. WebClient: reactive, non-blocking, fluent API. Supports sync and async. Use WebClient for new code — better for high concurrency. Add spring-webflux for WebClient even in non-reactive app